CHANDIGARH UNIVERSITY

Discover. Learn. Empower.

UNIVERSITY INSTITUTE OF COMPUTING

Project Report

Subject Name: Python Programming

Subject Code: 24CAH-301

Project Name: WEATHER DATA ANALYSIS SYSTEM

Submitted by:

Devansh Sood

24BCA10066

Section: 24BCA – 1 ‘A’

Submitted To:

Ms. Jyoti Rani

WEATHER DATA ANALYSIS SYSTEM

Mini Project Report — Weather Data Collection, Validation, Statistical Analysis, Forecasting & Alerting

Project Type: Academic Mini Project

Primary Language: Python 3.x (Standard Library)

Development IDE: Visual Studio Code (VS Code)

Execution State: Validated (End-to-End Pipeline Execution)

1. Abstract

The Weather Data Analysis System is a Python application that collects weather observations from several cities, validates the data, analyses it statistically, forecasts short-term temperature and raises weather alerts. Weather sensors in the real world often produce faulty readings caused by hardware errors or noise, and such readings lead to wrong conclusions if they are used without checking. This project therefore places a data-validation stage before any analysis is performed.

The system follows a five-stage pipeline: (1) simulated sensor data collection for five cities over seven days, (2) range-based validation that rejects impossible readings, (3) statistical analysis including average, minimum, maximum, standard deviation, Heat Index and Dew Point, (4) short-term temperature forecasting using a moving average with trend, and (5) rule-based alerting with export of results to CSV and JSON files. The whole project is built using only the Python standard library, so no external package needs to be installed.

2. Project Design & Implementation

The project follows a sequential pipeline architecture. Each stage is written as a separate class so that it can be understood, tested and modified independently:

Stage	Component Class	Responsibility	Output
1. Data Model	Reading	Stores one weather observation with its validity flags	Structured record
2. Collection	DataCollector	Simulates 7 days of readings (every 3 hours) for 5 cities	280 raw readings
3. Validation	DataValidator	Checks each value against physical limits and rejects faults	Clean list + QC counts
4. Analysis	WeatherAnalyzer	Statistics, Heat Index, Dew Point, forecast	City summaries
5. Alerts	AlertEngine	Applies threshold rules to the latest observation	Alert messages
6. Runner / Export	WeatherSystem	Runs the pipeline, prints report, writes files	CSV and JSON files

2.1 Monitoring Cities

Data is simulated for five cities with different climates. The base temperature and base humidity of each city are used together with a daily sine-wave cycle and random noise to produce realistic readings:

City	Base Temp (°C)	Base Humidity (%)	Climate Type
Ludhiana	27.0	55	Plains, moderate
Chandigarh	26.0	58	Plains, moderate
Amritsar	27.5	52	Plains, drier
Shimla	17.0	70	Hill station, cool and humid
Jaipur	30.0	45	Semi-arid, hot and dry

2.2 Data Validation (Quality Control)

Every reading is checked against physical limits before it is used. A reading that breaks any limit is flagged, marked invalid and excluded from the analysis, without stopping the program:

- **Temperature:** -50.0 °C to 55.0 °C
- **Humidity:** 0% to 100%
- **Pressure:** 850 hPa to 1090 hPa
- **Wind speed:** 0 to 250 km/h
- **Rainfall:** 0 to 300 mm

To test this stage, three faulty values are deliberately injected into the simulated data: a temperature of 95 °C (Jaipur), a humidity of 140% (Shimla) and a pressure of 700 hPa (Amritsar).

2.3 Mathematical Formulations

The system uses the following formulas:

- **1. Heat Index (Steadman / Rothfusz regression):** calculated only when temperature is at least 27 °C and relative humidity is at least 40%. Temperature is converted to Fahrenheit, a polynomial in T and RH is evaluated, and the result is converted back to Celsius.
- **2. Dew Point (Magnus-Tetens):** with constants a = 17.27 and b = 237.7, $\gamma = (a \cdot T)/(b+T) + \ln(RH/100)$ and Dew Point = $(b \cdot \gamma)/(a-\gamma)$.
- **3. Standard Deviation:** $\sqrt{\Sigma(x - \text{mean})^2 / n}$, used to measure how much temperature varies.
- **4. Short-term Forecast:** the mean of the last 4 readings plus the average recent trend multiplied by the number of steps ahead, giving forecasts for +3 h, +6 h and +9 h.

2.4 Alert Rules

Alert	Condition (latest observation)
HEAT ADVISORY	Heat Index ≥ 38 °C
HEAVY RAIN	Rainfall ≥ 8 mm
HIGH WIND	Wind speed ≥ 30 km/h
COLD ADVISORY	Temperature ≤ 5 °C

3. Python Code

The complete program is contained in `weather_system.py`. It implements the data model, data simulation, validation, analysis, forecasting, alerting and file export:

```
import math
import random
import json
import csv
import datetime
from typing import List, Dict, Any, Tuple

random.seed(66)  # reproducible demo run


# ============================================
# 1. DATA MODEL
# ============================================
class Reading:
    def __init__(self, city: str, timestamp: datetime.datetime, temp_c: float,
                 humidity_pct: float, pressure_hpa: float,
                 wind_kmh: float, rain_mm: float):
        self.city = city
        self.timestamp = timestamp
        self.temp_c = temp_c
        self.humidity_pct = humidity_pct
        self.pressure_hpa = pressure_hpa
        self.wind_kmh = wind_kmh
        self.rain_mm = rain_mm
        self.valid = True
        self.flags: List[str] = []

    def to_dict(self) -> Dict[str, Any]:
        return {
            "city": self.city,
            "timestamp": self.timestamp.strftime("%Y-%m-%d %H:%M"),
            "temp_c": round(self.temp_c, 2),
            "humidity_pct": round(self.humidity_pct, 1),
            "pressure_hpa": round(self.pressure_hpa, 1),
            "wind_kmh": round(self.wind_kmh, 1),
            "rain_mm": round(self.rain_mm, 1),
            "valid": self.valid,
            "flags": ";".join(self.flags),
        }


# ============================================
```

```
# 2. DATA COLLECTION (SIMULATED SENSORS)
# ========================================
class DataCollector:
    CITIES = [
        {"name": "Ludhiana", "base": 27.0, "hum": 55.0},
        {"name": "Chandigarh", "base": 26.0, "hum": 58.0},
        {"name": "Amritsar", "base": 27.5, "hum": 52.0},
        {"name": "Shimla", "base": 17.0, "hum": 70.0},
        {"name": "Jaipur", "base": 30.0, "hum": 45.0},
    ]

    @classmethod
    def generate(cls, days: int = 7) -> List[Reading]:
        readings = []
        start = datetime.datetime(2026, 10, 21, 0, 0)
        for d in range(days):
            for hour in range(0, 24, 3):                    # 8 readings/day
                ts = start + datetime.timedelta(days=d, hours=hour)
                cycle = math.sin((hour - 9) * math.pi / 12)   # daily temp cycle
                for c in cls.CITIES:
                    temp = c["base"] + 6 * cycle + random.uniform(-1.2, 1.2)
                    hum = c["hum"] - 12 * cycle + random.uniform(-4, 4)
                    hum = max(10.0, min(98.0, hum))
                    pres = 1010 + random.uniform(-4, 4)
                    wind = max(1.0, 10 + 5 * cycle + random.uniform(-4, 4))
                    rain = random.choice([0, 0, 0, 0, 0, 0, 1.5, 4.0, 9.0])
                    # injected faulty sensor values
                    if d == 2 and hour == 12 and c["name"] == "Jaipur":
                        temp = 95.0
                    if d == 4 and hour == 6 and c["name"] == "Shimla":
                        hum = 140.0
                    if d == 5 and hour == 18 and c["name"] == "Amritsar":
                        pres = 700.0
                    readings.append(Reading(c["name"], ts, temp, hum, pres, wind, rain))
        return readings


# ========================================
# 3. DATA CLEANING / VALIDATION
# ========================================
class DataValidator:
    LIMITS = {"temp": (-50.0, 55.0), "hum": (0.0, 100.0),
              "pres": (850.0, 1090.0), "wind": (0.0, 250.0), "rain": (0.0, 300.0)}

    @classmethod
    def validate(cls, data: List[Reading]) -> Tuple[List[Reading], Dict[str, int]]:
        clean = []
        stats = {"total": len(data), "valid": 0, "rejected": 0}
        L = cls.LIMITS
        for r in data:
            if not L["temp"][0] <= r.temp_c <= L["temp"][1]:
                r.flags.append("TEMP_OUT_OF_RANGE")
            if not L["hum"][0] <= r.humidity_pct <= L["hum"][1]:
                r.flags.append("HUMIDITY_OUT_OF_RANGE")
            if not L["pres"][0] <= r.pressure_hpa <= L["pres"][1]:
                r.flags.append("PRESSURE_OUT_OF_RANGE")
            if not L["wind"][0] <= r.wind_kmh <= L["wind"][1]:
                r.flags.append("WIND_OUT_OF_RANGE")
            if not L["rain"][0] <= r.rain_mm <= L["rain"][1]:
                r.flags.append("RAIN_OUT_OF_RANGE")
            r.valid = not r.flags
            if r.valid:
                clean.append(r)
                stats["valid"] += 1
            else:
                stats["rejected"] += 1
        return clean, stats


# ========================================
# 4. ANALYSIS ENGINE
# ========================================
class WeatherAnalyzer:
    @staticmethod
    def heat_index(temp_c: float, rh: float) -> float:
        if temp_c < 27.0 or rh < 40.0:
            return temp_c
        T = temp_c * 9 / 5 + 32
        hi = (-42.379 + 2.04901523 * T + 10.14333127 * rh - 0.22475541 * T * rh
              - 0.00683783 * T * T - 0.05481717 * rh * rh
              + 0.00122874 * T * T * rh + 0.00085282 * T * rh * rh
              - 0.00000199 * T * T * rh * rh)
        return (hi - 32) * 5 / 9
```

```
    @staticmethod
    def dew_point(temp_c: float, rh: float) -> float:
        a, b = 17.27, 237.7
        g = (a * temp_c) / (b + temp_c) + math.log(max(rh, 1.0) / 100.0)
        return (b * g) / (a - g)

    @staticmethod
    def std_dev(values: List[float]) -> float:
        m = sum(values) / len(values)
        return math.sqrt(sum((v - m) ** 2 for v in values) / len(values))

    @classmethod
    def city_summary(cls, data: List[Reading]) -> Dict[str, Dict[str, Any]]:
        groups: Dict[str, List[Reading]] = {}
        for r in data:
            groups.setdefault(r.city, []).append(r)
        out = {}
        for city, rs in groups.items():
            temps = [r.temp_c for r in rs]
            out[city] = {
                "readings": len(rs),
                "avg_temp_c": round(sum(temps) / len(temps), 2),
                "min_temp_c": round(min(temps), 2),
                "max_temp_c": round(max(temps), 2),
                "temp_std_dev": round(cls.std_dev(temps), 2),
                "avg_humidity_pct": round(sum(r.humidity_pct for r in rs) / len(rs), 1),
                "avg_dew_point_c": round(sum(cls.dew_point(r.temp_c, r.humidity_pct) for r in rs) / len(rs), 2),
                "max_heat_index_c": round(max(cls.heat_index(r.temp_c, r.humidity_pct) for r in rs), 2),
                "total_rain_mm": round(sum(r.rain_mm for r in rs), 1),
                "max_wind_kmh": round(max(r.wind_kmh for r in rs), 1),
            }
        return out

    @staticmethod
    def forecast(series: List[float], horizon: int = 3, window: int = 4) -> List[float]:
        """Moving average plus average recent trend."""
        if len(series) < window + 1:
            return [series[-1]] * horizon
        recent = series[-window:]
        avg = sum(recent) / window
        trend = (recent[-1] - recent[0]) / (window - 1)
        return [round(avg + trend * (i + 1), 2) for i in range(horizon)]

# ============================================
# 5. ALERT ENGINE
# ============================================
class AlertEngine:
    @staticmethod
    def evaluate(data: List[Reading]) -> List[str]:
        alerts = []
        latest_ts = max(r.timestamp for r in data)
        for r in (x for x in data if x.timestamp == latest_ts):
            hi = WeatherAnalyzer.heat_index(r.temp_c, r.humidity_pct)
            if hi >= 38:
                alerts.append(f"HEAT ADVISORY  - {r.city}: heat index {hi:.1f} C")
            if r.rain_mm >= 8:
                alerts.append(f"HEAVY RAIN     - {r.city}: {r.rain_mm:.1f} mm")
            if r.wind_kmh >= 30:
                alerts.append(f"HIGH WIND      - {r.city}: {r.wind_kmh:.1f} km/h")
            if r.temp_c <= 5:
                alerts.append(f"COLD ADVISORY  - {r.city}: {r.temp_c:.1f} C")
        return alerts


# ============================================
# 6. RUNNER / REPORT / EXPORT
# ============================================
class WeatherSystem:
    def run(self):
        line = "=" * 78
        print(line)
        print(">>> WEATHER DATA ANALYSIS SYSTEM <<<")
        print(line)

        print("[1/5] Collecting weather data for 5 cities (7 days)...")
        raw = DataCollector.generate(days=7)
        print(f"      [OK] Collected {len(raw)} readings")

        print("[2/5] Validating and cleaning data...")
        clean, qc = DataValidator.validate(raw)
        print(f"      [OK] Valid: {qc['valid']} | Rejected: {qc['rejected']}")
        for r in raw:
            if not r.valid:
```

```
                print(f"      [REJECTED] {r.city} {r.timestamp:%d-%b %H:%M} -> {', '.join(r.flags)}")

        print("[3/5] Computing statistics, heat index and dew point...")
        summary = WeatherAnalyzer.city_summary(clean)

        print("[4/5] Forecasting temperature and checking alerts...")
        alerts = AlertEngine.evaluate(clean)

        print("[5/5] Generating report...\n")
        print(line)
        print(f"{'CITY':<12}| {'AVG':>6} | {'MIN':>6} | {'MAX':>6} | {'STD':>5} | {'HUM%':>5} | {'DEW':>5} | {'RAIN':>6}")
        print("-" * 78)
        for city, m in summary.items():
            print(f"{city:<12}| {m['avg_temp_c']:>6.1f} | {m['min_temp_c']:>6.1f} | "
                  f"{m['max_temp_c']:>6.1f} | {m['temp_std_dev']:>5.2f} | "
                  f"{m['avg_humidity_pct']:>5.1f} | {m['avg_dew_point_c']:>5.1f} | {m['total_rain_mm']:>5.1f}mm")
        print("-" * 78)

        series = [r.temp_c for r in clean if r.city == "Ludhiana"]
        fc = WeatherAnalyzer.forecast(series, horizon=3)
        print("\nFORECAST (Ludhiana, next 3 readings = +3h, +6h, +9h):")
        print(f"  Latest: {series[-1]:.2f} C --> {fc[0]} C | {fc[1]} C | {fc[2]} C")

        print(f"\nACTIVE ALERTS ({len(alerts)}):")
        if not alerts:
            print("  [ALL CLEAR] No alerts for the latest observation.")
        for a in alerts:
            print("  ! " + a)
        print(line)

        self.export(clean, qc, summary, alerts)

    @staticmethod
    def export(clean, qc, summary, alerts):
        with open("weather_clean_data.csv", "w", newline="", encoding="utf-8") as f:
            w = csv.DictWriter(f, fieldnames=list(clean[0].to_dict().keys()))
            w.writeheader()
            for r in clean:
                w.writerow(r.to_dict())
        with open("weather_report.json", "w", encoding="utf-8") as f:
            json.dump({"generated_at": datetime.datetime.now().isoformat(),
                       "validation": qc, "city_summary": summary,
                       "alerts": alerts}, f, indent=4)
        print("\n[EXPORT COMPLETE] Saved 'weather_clean_data.csv' and 'weather_report.json'.\n")


if __name__ == "__main__":
    WeatherSystem().run()
```

4. Output / Screenshots

The program was run with `python weather_system.py`. The pipeline completed all five stages and printed the following dashboard (the random seed is fixed, so the same output is reproduced on every run):

```
================================================================================
>>> WEATHER DATA ANALYSIS SYSTEM <<<
================================================================================
[1/5] Collecting weather data for 5 cities (7 days)...
      [OK] Collected 280 readings
[2/5] Validating and cleaning data...
      [OK] Valid: 277 | Rejected: 3
      [REJECTED] Jaipur 23-Oct 12:00 -> TEMP_OUT_OF_RANGE
      [REJECTED] Shimla 25-Oct 06:00 -> HUMIDITY_OUT_OF_RANGE
      [REJECTED] Amritsar 26-Oct 18:00 -> PRESSURE_OUT_OF_RANGE
[3/5] Computing statistics, heat index and dew point...
[4/5] Forecasting temperature and checking alerts...
[5/5] Generating report...

================================================================================
CITY        |    AVG |    MIN |    MAX |   STD |  HUM% |   DEW |   RAIN
--------------------------------------------------------------------------------
Ludhiana    |   27.1 |   19.9 |   34.2 |  4.40 |  55.1 |  17.1 |  56.5mm
Chandigarh  |   26.0 |   19.1 |   33.0 |  4.33 |  58.0 |  16.8 | 104.5mm
Amritsar    |   27.4 |   20.4 |   34.5 |  4.27 |  52.0 |  16.3 |  88.0mm
Shimla      |   17.1 |   10.0 |   24.1 |  4.46 |  69.6 |  11.3 | 105.0mm
Jaipur      |   30.0 |   22.9 |   37.0 |  4.46 |  44.7 |  16.2 | 105.5mm
--------------------------------------------------------------------------------

FORECAST (Ludhiana, next 3 readings = +3h, +6h, +9h):
  Latest: 27.42 C --> 29.47 C | 28.38 C | 27.29 C

ACTIVE ALERTS (3):
  ! HEAVY RAIN     - Chandigarh: 9.0 mm
  ! HEAVY RAIN     - Amritsar: 9.0 mm
  ! HEAVY RAIN     - Jaipur: 9.0 mm
================================================================================

[EXPORT COMPLETE] Saved 'weather_clean_data.csv' and 'weather_report.json'.
```

4.1 Observations from the Output

- Of 280 readings, 277 were accepted and 3 were rejected, which matches the three faults injected for testing.
- Jaipur is the hottest city (average 30.0 °C) and Shimla the coolest (average 17.1 °C), as expected from their base temperatures.
- The dew point is well below the air temperature in every city, showing that the air is not saturated on average.
- Three heavy-rain alerts (9.0 mm each) were raised for Chandigarh, Amritsar and Jaipur from the latest observation.

4.2 Exported Data Files

- **weather_clean_data.csv:** 277 validated observation rows with the fields city, timestamp, temp_c, humidity_pct, pressure_hpa, wind_kmh, rain_mm, valid and flags.
- **weather_report.json:** machine-readable report containing the generation time, validation counts, per-city statistics and the active alerts.

5. Conclusion

The Weather Data Analysis System shows how a complete data pipeline can be built in Python using only the standard library. Validating the data before analysis kept faulty sensor values out of the results, and the statistics, forecast and alerts are produced automatically. Future work could connect the system to a real weather API, use more advanced forecasting such as Holt-Winters smoothing, and add graphs for visualisation.

6. GitHub Repository Link

Project Repository & Source Control

GitHub Repository URL: ***[add your repository link here]***